

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
sales_performance= pd.read_csv('/content/sales_performance.csv', encoding='latin1')
```

```
sales_performance.head()
```

	Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	Country	
0	7981	CA-2015-103800	03-01-2015	07-01-2015	Standard Class	DP-13000	Darren Powers	Consumer	United States	Hou
1	740	CA-2015-112326	04-01-2015	08-01-2015	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Nape
2	741	CA-2015-112326	04-01-2015	08-01-2015	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Nape
3	742	CA-2015-112326	04-01-2015	08-01-2015	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Nape
4	1760	CA-2015-141817	05-01-2015	12-01-2015	Standard Class	MB-18085	Mick Brown	Consumer	United States	Philadel

```
sales_performance["Postal_Code"] = sales_performance["Postal_Code"].fillna(0)
```

```
sales_performance["Order_Date"] = pd.to_datetime(sales_performance["Order_Date"], format='%d-%m-%Y')
sales_performance["Ship_Date"] = pd.to_datetime(sales_performance["Ship_Date"], format='%d-%m-%Y')
sales_performance.dtypes
```

0	
Row_ID	int64
Order_ID	object
Order_Date	datetime64[ns]
Ship_Date	datetime64[ns]
Ship_Mode	object
Customer_ID	object
Customer_Name	object
Segment	object
Country	object
City	object
State	object
Postal_Code	float64
Region	object
Product_ID	object
Category	object
Sub_Category	object
Product_Name	object
Sales	float64

dtype: object

```
df["Year"] = df["Order_Date"].dt.year
df["Month"] = df["Order_Date"].dt.month_name()
df["Quarter"] = df["Order_Date"].dt.quarter
df["Processing Days"] = (df["Ship_Date"] - df["Order_Date"]).dt.days
```

```
total_sales = df["Sales"].sum()
print("Total Revenue:", total_sales)
```

Total Revenue: 2261536.7827

```
print(df.columns)
```

```
Index(['Row_ID', 'Order_ID', 'Order_Date', 'Ship_Date', 'Ship_Mode',
      'Customer_ID', 'Customer_Name', 'Segment', 'Country', 'City', 'State',
      'Postal_Code', 'Region', 'Product_ID', 'Category', 'Sub_Category',
      'Product_Name', 'Sales', 'Year', 'Month', 'Quarter', 'Processing Days'],
      dtype='object')
```

```
sales_performance["Cost"] = sales_performance["Sales"] * 0.70
```

```
sales_performance["Profit"] = sales_performance["Sales"] - sales_performance["Cost"]
```

```
sales_performance.head()
```

	Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	Country	
0	7981	CA-2015-103800	03-01-2015	07-01-2015	Standard Class	DP-13000	Darren Powers	Consumer	United States	Hou
1	740	CA-2015-112326	04-01-2015	08-01-2015	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Nape
2	741	CA-2015-112326	04-01-2015	08-01-2015	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Nape
3	742	CA-2015-112326	04-01-2015	08-01-2015	Standard Class	PO-19195	Phillina Ober	Home Office	United States	Nape
4	1760	CA-2015-141817	05-01-2015	12-01-2015	Standard Class	MB-18085	Mick Brown	Consumer	United States	Philadel

```
sales_performance["Sales"].dtype
```

```
dtype('float64')
```

```
sales_performance["Sales"] = pd.to_numeric(sales_performance["Sales"], errors="coerce")
```

```
sales_performance["Cost"] = sales_performance["Sales"] * 0.70
sales_performance["Profit"] = sales_performance["Sales"] - sales_performance["Cost"]
```

```
sales_performance[["Sales", "Cost", "Profit"]].head()
```

	Sales	Cost	Profit
0	16.448	11.5136	4.9344
1	11.784	8.2488	3.5352
2	272.736	190.9152	81.8208
3	3.540	2.4780	1.0620
4	19.536	13.6752	5.8608

```
sales_performance["Profit"].sum()
```

```
np.float64(678461.0348100001)
```

```
print(sales_performance["Sales"].sum())
```

```
2261536.7827
```

```
total_sales = sales_performance["Sales"].sum()
total_profit = sales_performance["Profit"].sum()

profit_margin = (total_profit / total_sales) * 100

print("Profit Margin:", profit_margin)
```

```
Profit Margin: 30.000000000000004
```

```
average_order_value = df.groupby("Order_ID")["Sales"].sum().mean()
```

```
print("Average Order Value:", round(average_order_value,2))
```

```
Average Order Value: 459.48
```

```
total_orders = df["Order_ID"].nunique()
```

```
print("Total Orders:", total_orders)
```

```
Total Orders: 4922
```

```
customers = df["Customer_ID"].nunique()
```

```
print("Total Customers:", customers)
```

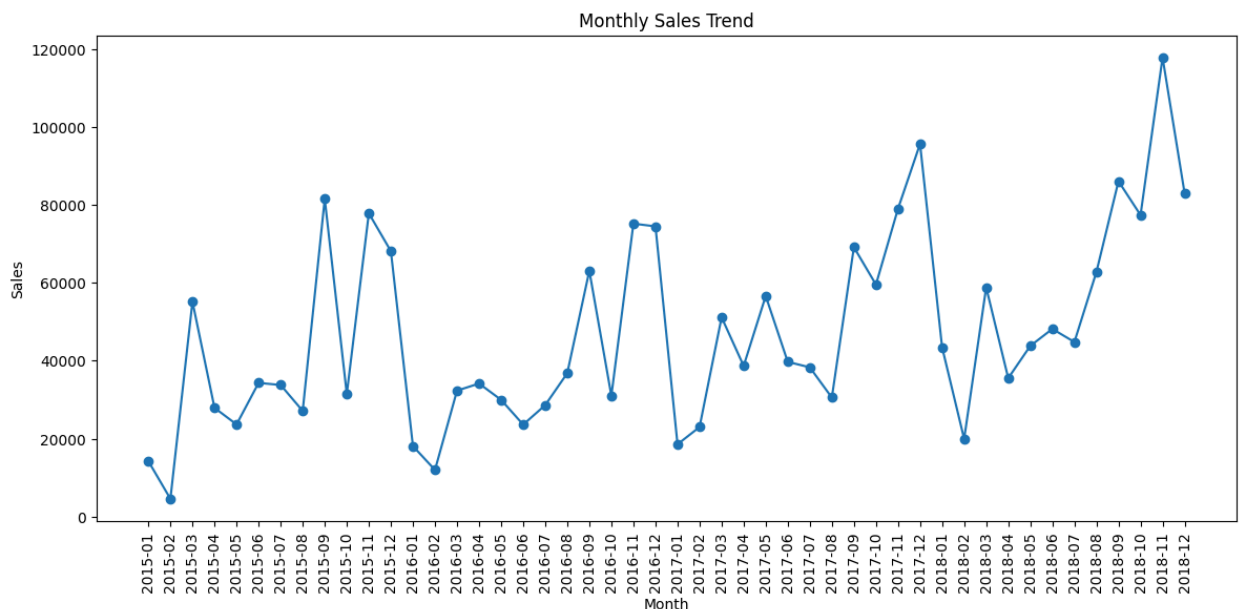
```
Total Customers: 793
```

sales trend

```
monthly_sales = df.groupby(df["Order_Date"].dt.to_period("M"))["Sales"].sum()
```

```
monthly_sales.index = monthly_sales.index.astype(str)
```

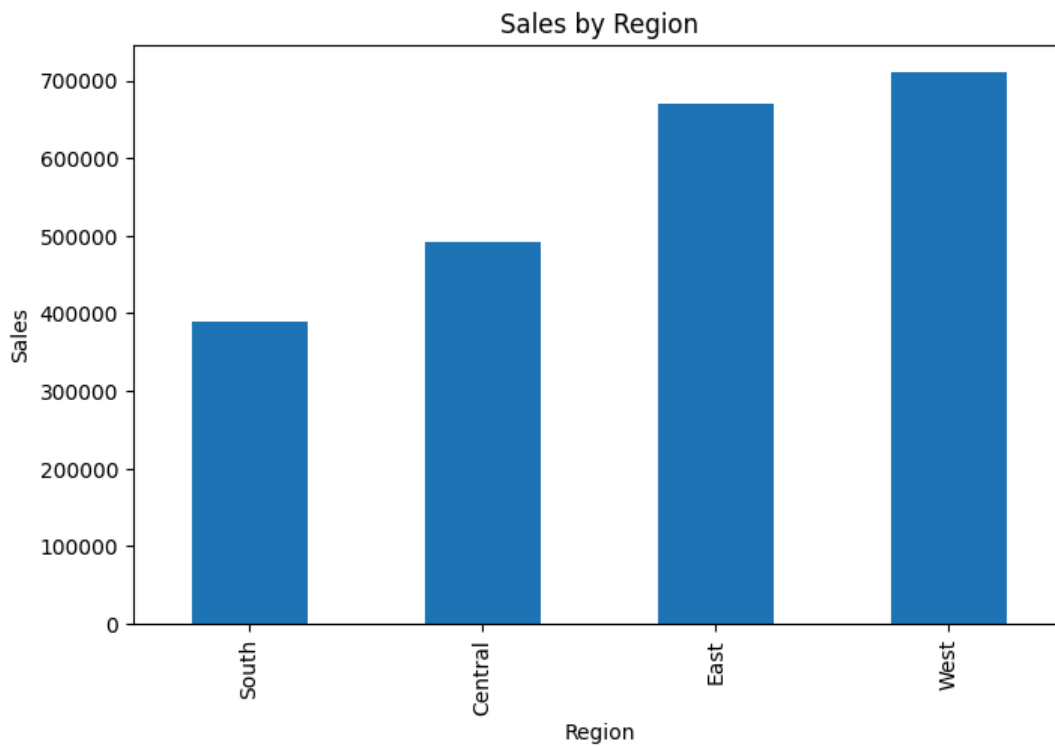
```
plt.figure(figsize=(14,6))
plt.plot(monthly_sales.index, monthly_sales.values, marker='o')
plt.xticks(rotation=90)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```



regionals performance

```
region_sales = df.groupby("Region")["Sales"].sum().sort_values()
```

```
plt.figure(figsize=(8,5))
region_sales.plot(kind="bar")
plt.title("Sales by Region")
plt.ylabel("Sales")
plt.show()
```



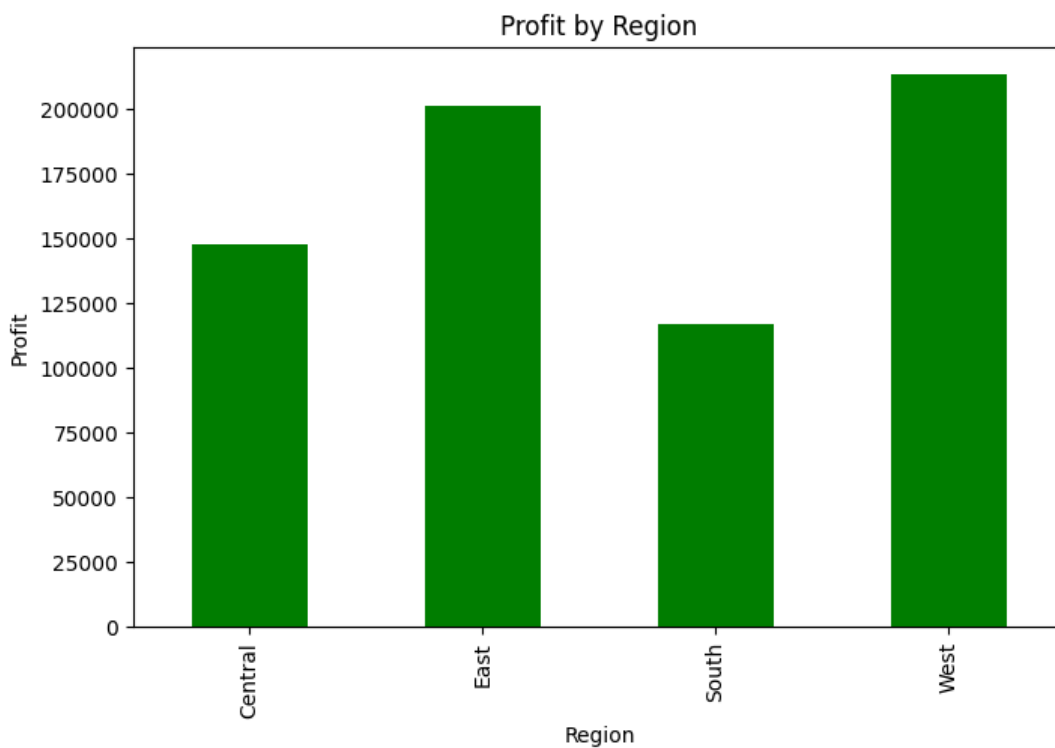
Profit by Region

```
region_profit = sales_performance.groupby("Region")["Profit"].sum()

plt.figure(figsize=(8,5))
region_profit.plot(kind="bar", color="green")

plt.title("Profit by Region")
plt.xlabel("Region")
plt.ylabel("Profit")

plt.show()
```



Create Cost and Profit

```

sales_performance["Sales"] = sales_performance["Sales"].astype(str)

sales_performance["Sales"] = sales_performance["Sales"].str.replace(",","")

sales_performance["Sales"] = sales_performance["Sales"].str.replace("₹"," ", regex=False)

sales_performance["Sales"] = pd.to_numeric(sales_performance["Sales"], errors="coerce")

```

```

sales_performance["Cost"] = sales_performance["Sales"] * 0.70

sales_performance["Profit"] = sales_performance["Sales"] - sales_performance["Cost"]

```

```
sales_performance[["Sales","Cost","Profit"]].head()
```

	Sales	Cost	Profit
0	16.448	11.5136	4.9344
1	11.784	8.2488	3.5352
2	272.736	190.9152	81.8208
3	3.540	2.4780	1.0620
4	19.536	13.6752	5.8608

```

total_sales = sales_performance["Sales"].sum()
total_profit = sales_performance["Profit"].sum()

profit_margin = (total_profit / total_sales) * 100

average_order_value = (
    sales_performance.groupby("Order_ID")["Sales"].sum().mean()
)

print("Total Sales:", round(total_sales,2))
print("Total Profit:", round(total_profit,2))
print("Profit Margin:", round(profit_margin,2), "%")
print("Average Order Value:", round(average_order_value,2))

```

```

Total Sales: 2261536.78
Total Profit: 678461.03
Profit Margin: 30.0 %
Average Order Value: 459.48

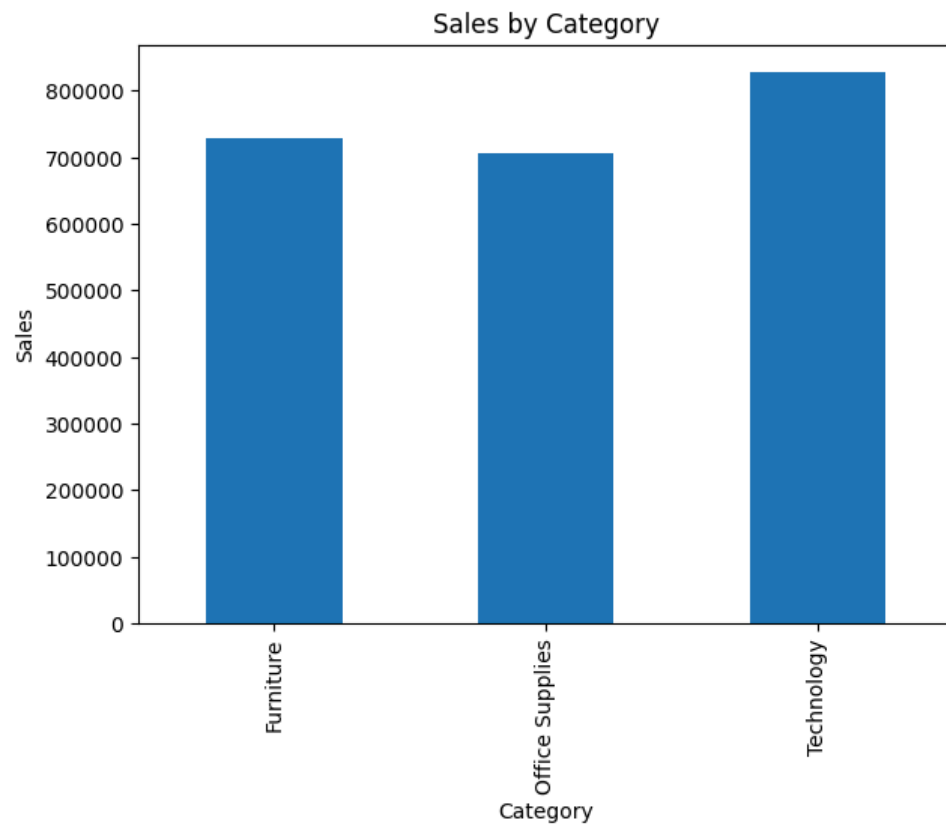
```

```

category_sales = sales_performance.groupby("Category")["Sales"].sum()

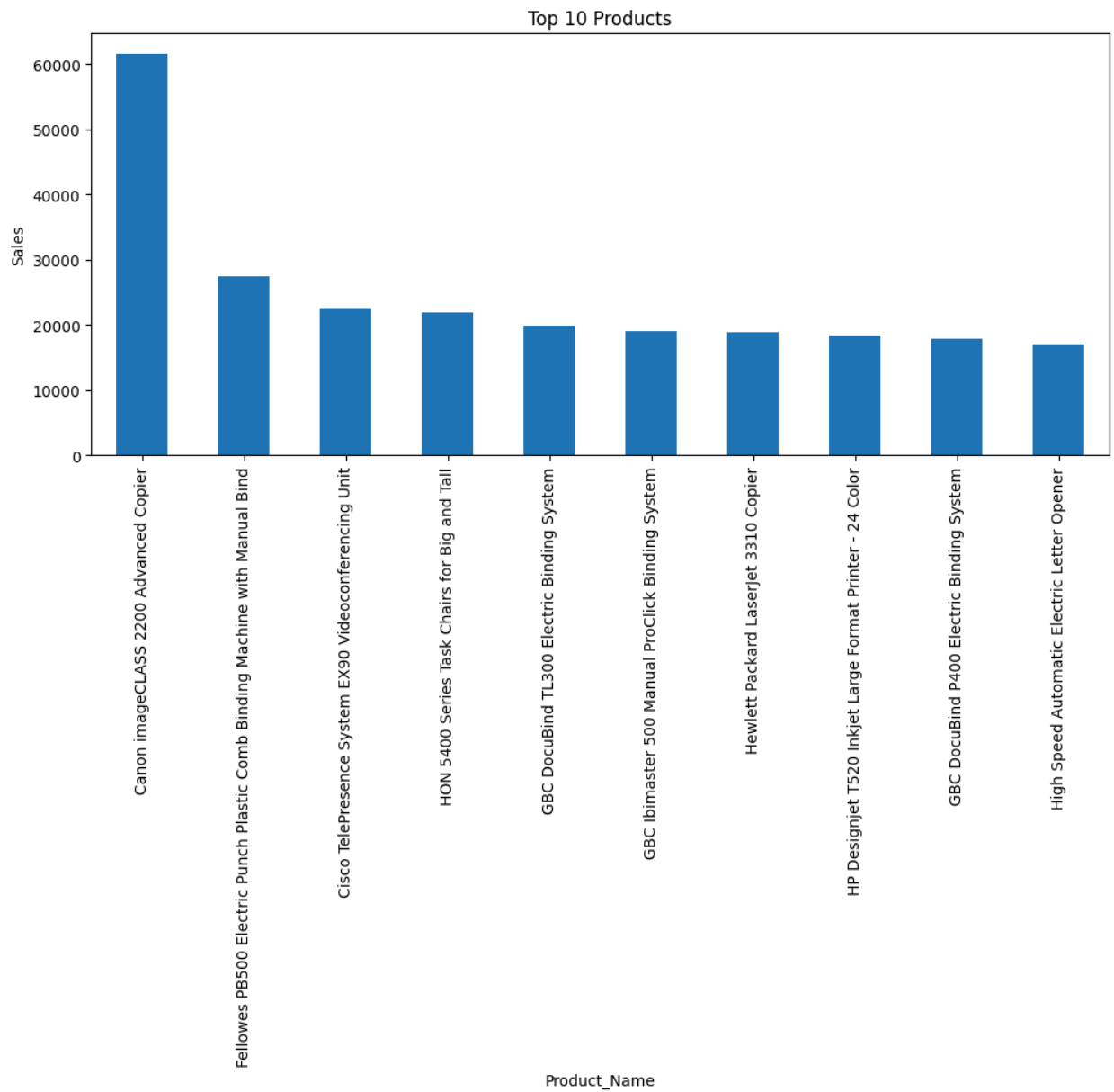
plt.figure(figsize=(7,5))
category_sales.plot(kind="bar")
plt.title("Sales by Category")
plt.ylabel("Sales")
plt.show()

```



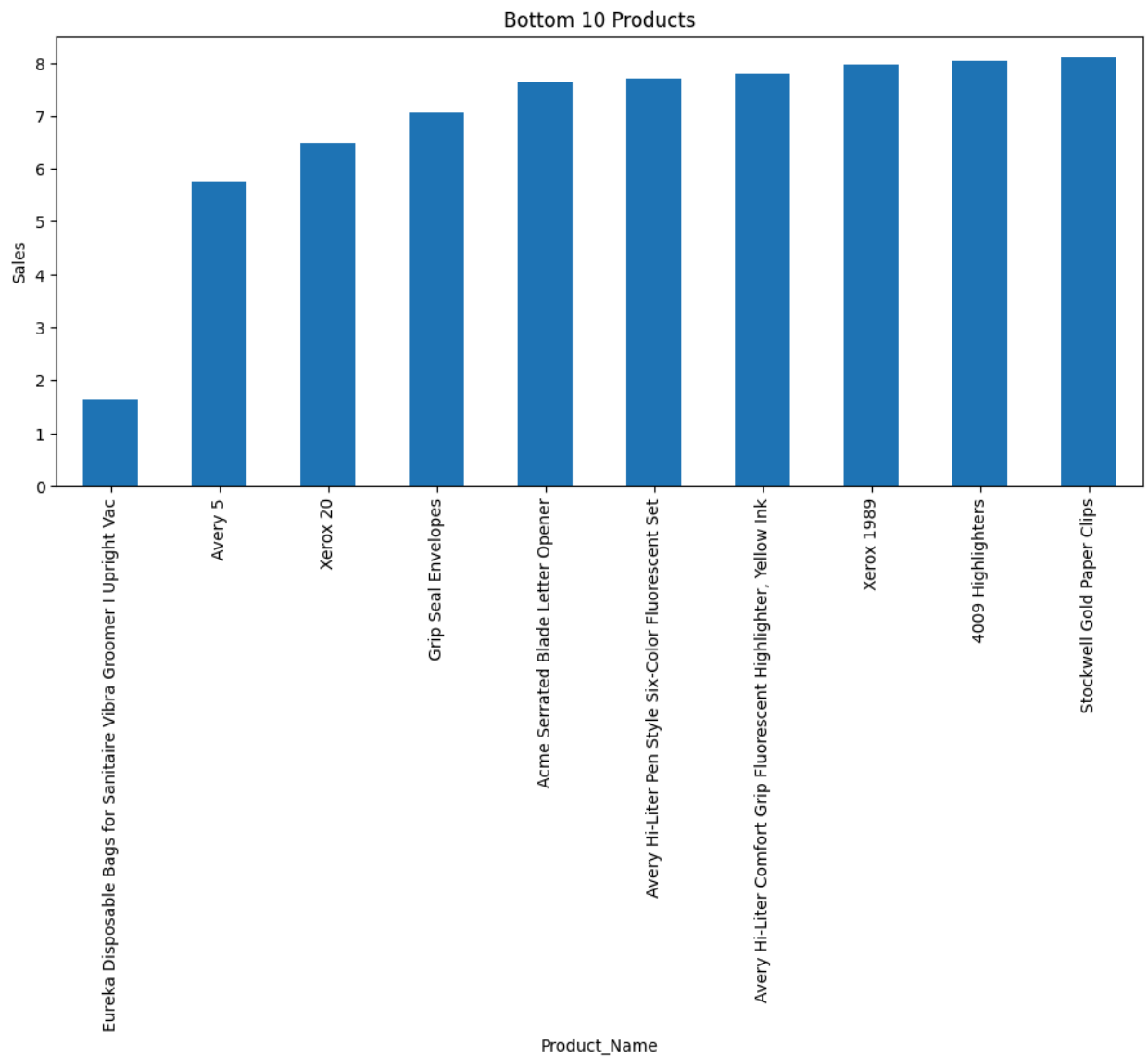
```
top_products = sales_performance.groupby("Product_Name")["Sales"].sum().sort_values(ascending=False).head(10)

plt.figure(figsize=(12,5))
top_products.plot(kind="bar")
plt.title("Top 10 Products")
plt.ylabel("Sales")
plt.show()
```



```
bottom_products = sales_performance.groupby("Product_Name")["Sales"].sum().sort_values().head(10)

plt.figure(figsize=(12,5))
bottom_products.plot(kind="bar")
plt.title("Bottom 10 Products")
plt.ylabel("Sales")
plt.show()
```

```
segment_sales = sales_performance.groupby("Segment")["Sales"].sum()

plt.figure(figsize=(6,6))
plt.pie(segment_sales, labels=segment_sales.index, autopct="%1.1f%%")
plt.title("Sales by Segment")
plt.show()
```

Sales by Segment

Consumer

discount column is not exist in data

50.8%

Double-click (or enter) to edit

```
top_states = df.groupby("State")["Sales"].sum().sort_values(ascending=False).head(10)
```

```
plt.figure(figsize=(10,5))
top_states.plot(kind="bar")
plt.title("Top States by Sales")
plt.ylabel("Sales")
plt.show()
```

